

README File for Code Design Peer Review

Class: *MapGraph*

Modifications made to MapGraph (what and why):

Class added:

- Class name: **Road**
- Purpose and description of class:
 - This class is added as an *inner* class inside the *MapGraph* class.
 - This class represents a road (corresponding to an edge) in between two intersection points.
 - This class will be *private* and only accessible to the *MapGraph* class.
 - This class will store the information (such as name, type, length) about a road connecting two intersections, which will be stored as private member variables.
 - This class will also contain public getter methods to access its member variables (will be useful in future).
 - A new *instance* of a road class (a road *object*) can only be created using a 3-argument constructor that will provide all information required to create a road.
 - The class is shown below.

roadgraph

Class MapGraph.Road

```
java.lang.Object
roadgraph.MapGraph.Road
```

Enclosing class:
MapGraph

```
private class MapGraph.Road
extends java.lang.Object
```

A class which represents a road in between two intersections

Field Summary

Fields

Modifier and Type	Field and Description
private double	length
private java.lang.String	name
private java.lang.String	type

Constructor Summary

Constructors

Constructor and Description
Road(java.lang.String name, java.lang.String type, double length) Constructor to create a new road

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method and Description
double	getLength()
java.lang.String	getName()
java.lang.String	getType()

Member Variables added:

1. **numVertices**: to store the *number of vertices* in the *MapGraph*, to return the value stored in this variable when asked for.
2. **numEdges**: to store the *number of edges* in the *MapGraph*, to return the value stored in this variable when asked for.
3. **adjListsMap**: the *MapGraph* is stored as *adjacency list* representation and this variable will store the adjacency list.
4. **roadMap**: to store all the roads in between the pairs of nodes (intersection points) that have an edge in between them.

Overall Design Justification

1. The *MapGraph* is stored as *adjacency list*, since it will need much less space (because the *MapGraph* in general will be sparse - not every pair of nodes will have an edge between them).
2. For each edge in the *MapGraph* we need to have a corresponding *road*. Each road should also have its name, type and length information associated with it. So, it makes sense to create a new class *Road* altogether (*abstracted out*) that will have those information as its member variables.
3. This *Road* class needs to be only accessed (and *encapsulated*) by the *MapGraph* class and the outer world need not know about this, hence it's *private* to the *MapGraph* class. This *road* class also contains public getter methods to access its private member variables (will be useful in future).
4. Also, there needs to be a *one-to-one* correspondence between the roads and edges in the *MapGraph*. This will be accomplished by the map *roadMap* which will store the road object created in between two intersection points (nodes) when a new edge between those two nodes gets created.